

Understanding and Predicting Accepted Code Suggestions in AI-Assisted Programming

JING JIANG, SKLCCSE, Beihang University, China
LIEHAO LI, SKLCCSE, Beihang University, China
JINYUN HOU, SKLCCSE, Beihang University, China
XIN TAN*, SKLCCSE, Beihang University, China
LI ZHANG, SKLCCSE, Beihang University, China

AI-assisted programming tools are widely adopted, yet their practical utility is often undermined by undesired suggestions that interrupt developer workflows and cause frustration. While existing research has explored developer-AI interactions when programming qualitatively, a significant gap remains in quantitative analysis of developers' acceptance of AI-generated code suggestions, partly because the necessary fine-grained interaction data is often proprietary. To bridge this gap, this paper conducts an empirical study using 66,239 industrial developer-AI interactions from a large technology company. We analyze features that are significantly different between accepted code suggestions and rejected ones. We find that accepted suggestions are characterized by significantly higher historical acceptance counts and ratios for both developers and projects, longer generation intervals, shorter preceding code context in the project, and older IDE versions. Based on these findings, we introduce *CSAP* (Code Suggestion Acceptance Prediction) to predict whether a developer will accept the code suggestion before it is displayed. Our evaluation of *CSAP* shows that it achieves the accuracy of 0.973 and 0.922 on imbalanced and balanced dataset respectively. Compared to a large language model baseline and an in-production industrial filter, *CSAP* relatively improves the accuracy by 12.6% and 69.5% on imbalanced dataset, and improves the accuracy by 87.0% and 140.1% on balanced dataset. Our results demonstrate that targeted personalization is a powerful approach for filtering out code suggestions with predicted rejection and reduce developer interruption. To the best of our knowledge, it is the first quantitative study of code suggestion acceptance on large-scale industrial data, and this work also sheds light on an important research direction of AI-assisted programming.

CCS Concepts: • **Software and its engineering** → **Automatic programming**.

Additional Key Words and Phrases: Accepted Code Suggestion, AI-Assistant Programming, Code Generation

1 Introduction

AI-assisted programming tools powered by large language models, such as GitHub Copilot and Cursor, are aimed at providing programmers with code suggestions to help improve their productivity [4, 12, 32, 46]. These systems usually operate by displaying the suggestion as ghost text—a grayed-out code suggestion inline inside the IDE, which programmers can accept or reject. However, not all the code suggestions generated by these state-of-the-art systems are helpful

*Corresponding author.

SKLCCSE stands for State Key Laboratory of Complex and Critical Software Environment.

Authors' Contact Information: Jing Jiang, SKLCCSE, Beihang University, Beijing, China, jiangjing@buaa.edu.cn; Liehao Li, SKLCCSE, Beihang University, Beijing, China, liehao@buaa.edu.cn; Jinyun Hou, SKLCCSE, Beihang University, Beijing, China, zy2206235@buaa.edu.cn; Xin Tan, SKLCCSE, Beihang University, Beijing, China, xintan@buaa.edu.cn; Li Zhang, SKLCCSE, Beihang University, Beijing, China, lily@buaa.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE118

<https://doi.org/10.1145/3808125>

to the developers. For instance, one study on GitHub Copilot found that developers rejected approximately 70% of the suggestions shown to them [47]. This high rate of unhelpful suggestions poses a significant threat to the primary goal of these tools. When a suggestion is rejected, it can disrupt a developer's cognitive flow, forcing them to pause their work to review and discard the irrelevant code, which wastes valuable time and leads to frustration [6, 15, 17, 24, 35, 39]. This highlights a critical need to understand the underlying drivers of suggestion acceptance.

The difference between accepted code suggestions and rejected code suggestions is not yet well understood. While existing research has explored developer interaction with these tools qualitatively [6, 26, 27, 33, 37, 38], a significant gap remains in the quantitative analysis of suggestion acceptance. One reason is that much of the fine-grained interaction data resides within company-internal environments and is not publicly accessible. Moreover, the log data that can typically be captured from IDEs—such as acceptance or rejection events—are insufficient on their own to explain or predict developers' decisions. Consequently, the problem of predicting the acceptance of AI-generated code suggestions remains a relatively underexplored area of research. Without acceptance prediction for code suggestions, AI-assisted programming tools directly provide code suggestions to developers, and some suggestions are rejected, which may negatively impact the developer experience and reduce their willingness to use the code completion tool. Therefore, building a model to predict the acceptance of code suggestions is crucial for optimizing developer-AI collaboration in software engineering. If code suggestions are predicted to be rejected, they can be removed without disrupting developers.

In this paper, we conduct an empirical study about accepted code suggestions. We collect 66,239 developer-AI interaction logs from our industry partner, encompassing every accepted and rejected suggestion, along with associated code context, developer characteristics, and project environments. Based on this company-internal dataset, we posit the following research questions:

- **RQ1: What are the significant features of the accepted code suggestions?**

Based on previous works and information provided by our industry partner, we design candidate features from three dimensions: developer habit, project preference, and in-situ context. Developer habit features and project preference features analyze historical characteristics of a developer or a project, respectively. In-situ context features capture a snapshot of the immediate circumstances surrounding a single code suggestion. We perform a significance analysis and identify important features for which accepted code suggestions and rejected code suggestions have significantly different values. We find that: At the developer level, accepted code suggestions are characterized by the developer's significantly higher historical acceptance count and ratio, and longer generation intervals; At the project level, they are characterized by the project's significantly higher historical acceptance count and ratio, and a shorter preceding code context; Furthermore, accepted code suggestions have older IDE versions.

- **RQ2: How can we design an effective prediction method to determine whether a developer will accept a code suggestion?**

Based on features which are significantly different between accepted and rejected code suggestions in RQ1, we perform the correlation analysis and derive the feature set for prediction. We use a simple neural network with class-balanced binary cross-entropy loss function to build CSAP (Code Suggestion Acceptance Prediction), and predict whether a developer will accept a code suggestion before it is displayed. The evaluation of CSAP shows that it achieves the accuracy of 0.973 and 0.922 on imbalanced and balanced dataset respectively. Compared to the direct LLM call (Qwen2.5-Coder-32B) and the Circuit Breaker model (in-production industrial filter), CSAP improves the accuracy by 12.6% and 69.5% on imbalanced dataset, and improves the accuracy by 87.0% and 140.1% on balanced dataset. Feature analysis reveals that the most critical predictors, stemming from in-situ

context, developer habits, are the *in-situ_IDE_version*, and *developer_accepted_ratio*. These features highlight the importance of tailoring predictions to a developer's immediate environment and past behavior.

This paper makes the following contributions:

- We conduct a first large-scale quantitative study using fine-grained enterprise IDE interaction data and identify important features of accepted code suggestions.
- We propose and evaluate *CSAP*, a personalized code acceptance prediction method, which can be used to filter out code suggestions with rejected prediction and reduce developer interruption.

2 BACKGROUND AND RELATED WORK

2.1 AI-Assisted Programming

2.1.1 Developer Needs in AI-Assisted Programming. The recent proliferation of large language models [10, 16, 43] has spurred the development of powerful AI-assisted programming tools that leverage these models for code generation [2, 14, 25, 45]. Consequently, understanding developer needs has become a critical area of research to ensure these tools are effective and well-adopted. A study by Davila et al. [11] shows that a primary motivation for using these tools is to write better or alternative solutions to programming problems. However, effectiveness extends beyond just the quality of the output. To maintain a seamless development workflow, it is crucial that these tools avoid disruption, for instance, by filtering out low-confidence suggestions as argued by Ciniselli et al. [9]. Furthermore, to truly accelerate the development process, developers expect generated code to align with their personal coding styles and preferences [7–9, 22, 26, 27, 40].

2.1.2 Factors Influencing Developers' Acceptance of Code Suggestions. As AI-assisted programming tools proliferate, a growing body of research has examined the developer-experience determinants of developers' acceptance of AI-generated code suggestions. Specifically, regarding Code Factors, prior studies by Long et al. [27] and Liang et al. [26] found that code complexity is associated with rejections. Furthermore, Barke et al. [6] noted that developers in "acceleration" mode prefer concise code, whereas those in "exploration" mode value code with comments to guide their thinking. Motivated by these findings, we designed complexity-related features (e.g., considering comment length and code length) to serve as proxies for code complexity. Regarding Developer Factors, Ziegler et al. [47] highlighted the correlation between suggestion acceptance and working hours. Therefore, we develop time-related features (e.g., considering working period) to capture the developer's work state. Collectively, these studies provide qualitative foundations for understanding acceptance behavior. However, large-scale quantitative evidence, especially fine-grained in-IDE telemetry that captures real decision signals, remains limited. This gap motivates our study, which utilizes enterprise developer-AI interaction logs to quantify which factors are significantly different between accepted code suggestions and rejected code suggestions.

2.2 Predictive Modeling of Developer Decision

Predictive modeling of developer behavior is a long-standing topic in software engineering [5, 20, 30, 31]. By contrast, work that directly predicts whether developers will accept AI-generated code suggestions remains scarce, in part because enterprise IDE deployments rarely expose fine-grained, real-time usage logs of developers' coding actions that record actual acceptance decisions.

Relevant studies have investigated filtering mechanisms based on quality estimation. Li et al. [24] utilized coding simulations to build acceptance models that depend on model confidence scores. Sun et al. [36] designed early-rejection estimators specifically for function-level tasks, utilizing synthetically constructed datasets for training. These works have significantly advanced

the understanding of how to reduce the hidden costs of inspecting unhelpful suggestions. However, these works do not consider developer personalization, focusing solely on code context features. Furthermore, we extend our scope beyond function-level tasks to encompass all scenarios where code suggestions are triggered.

We also draw on adjacent research predicting pull-request (PR) outcomes [3, 18, 19, 21]. While these works target asynchronous workflows rather than real-time IDE interactions, their feature-design principles—combining artifact, developer, and process signals—inform our selection of code-, developer-, and project-level features and our quantitative evaluation.

3 RQ1: Significant Features of the Accepted Code Suggestions

In this section, leveraging data from our industry partner, we identify significant features of the accepted code suggestions. Specifically, in RQ1 we: (1) describe the rationale of feature design; (2) propose candidate features; and (3) test their significance to find features which accepted code suggestions and rejected code suggestions have significantly different values.

3.1 Rationale of Feature Design

3.1.1 Circuit Breaker (abbreviated as Cbk). This study is conducted in partnership with a leading global technology company specializing in software and systems development. We investigate their internally deployed, LLM-driven IDE, which provides real-time code completion and generation. This AI-assisted programming tool integrates with mainstream IDEs like VS Code and has seen significant adoption, with over 10,000 installations across numerous engineering teams. This large-scale deployment provides the real-world setting for our study. AI-generated code suggestions are automatically displayed as grayed-out ghost text during typing and can be received by developers with a single keystroke (e.g., Tab), which can be further modified by developers. While the tool proactively presents these suggestions, the decision to accept or ignore it is entirely developer-driven, and this is the behavior we model.

Notably, our industry partner has deployed an in-use acceptance prediction method, called Circuit Breaker, which uses numerical features to predict accepted code suggestions, such as the number of preceding lines of code. We take the input features of this method as a reference when identifying candidate features.

3.1.2 Developer Feedback (abbreviated as Fdbk). Our industry partner provides us with some feedback records, which inspire us to design features. Our industry partner provides 25 feedback records between April 22–May 31, 2024, which include developers’ experience and feedback towards the AI-assisted programming tool.

3.1.3 AI-Developer Interaction Logs (abbreviated as Log). Most importantly, our industry partner provides detailed developer-AI interaction logs within the IDE from developers with varying skill levels across multiple projects. According to developer-AI interaction logs, we utilize available data to design features. The main components of each log are detailed as follows:

- **Preceding Context:** refers to the content within the current file that exists before the cursor position at the trigger point (i.e., the line or character position), serving as the contextual input for the code generation model.
- **Generated Code Suggestions:** refers to the code suggestions generated by the code generation model.
- **Max Generation Length:** refers to the maximum lines and characters of the generated code suggestions set by the developer.

- **Written Code:** refers to the code written between consecutive generation triggers within the same file.
- **Modified Code:** refers to any modifications applied to an accepted code suggestion before the next generation trigger.
- **IDE Type:** refers to the specific name of the IDE (e.g., VS Code).
- **IDE Version:** refers to the specific release number of the IDE, namely a proxy of time for the IDE version installed.
- **Generation Time for Code Suggestions:** refers to the total duration from the moment a code suggestion is requested (e.g., by typing or pausing) to the moment the generated suggestions are fully displayed in the IDE.
- **Trigger Timestamp:** refers to the time point when the code generation function is triggered.
- **Accepted Length Ratio:** refers to the ratio of the accepted code length to the total length of the generated suggestion. Unlike a boolean acceptance status, this metric captures partial adoption where developers only retain a specific portion of the suggestion.
- **Acceptance Results:** refers to a boolean value indicating whether a generated code suggestion is accepted by the developer.

3.1.4 *Literature References (abbreviated as Ref).* To inform our feature design, we draw on prior work in several areas. This includes studies on pull request acceptance [42, 44], research on programmer expectations and interaction with AI tools [9, 13, 34], and analysis of how the coding environment impacts developer behavior [45, 47].

3.2 Candidate Features

We design a compact feature set which spans three dimensions: developer habit, project preference, and in-situ context. Developer habit features and project preference features analyze historical characteristics of a developer or a project, respectively. In-situ context features capture a snapshot of the immediate circumstances surrounding a single code suggestion.

3.2.1 *Developer Habit Features.* Developer habit features capture behavior patterns and preferences by analyzing a developer’s recent coding history. To determine the optimal length of this history, we evaluated 7, 30, and 60-day windows and found that the shorter 7-day window identified the same set of significant features. We adopt this shorter time window to improve computational efficiency, and the detailed comparison results are available in our replication package. As shown in Table 1, these features are further divided into three aspects, including developer code features, developer acceptance preference features and developer coding environment features.

Developer Code Features:

- **Preceding Context length.** Our industry partner’s Circuit Breaker uses context length to help predict acceptance. Inspired by this, we investigate how context length, measured in both lines and characters, may influence a developer’s decision to accept a generated code suggestion. To investigate this, we characterize the length of a suggestion’s preceding context by measuring the average number of lines (*developer_preceding_lines*) and characters (*developer_preceding_chars*) for a given developer.
- **Generated Code Length.** In preliminary developer feedback, a developer suggests a feature to automatically wrap single-line completions that exceed 120 characters, noting that code with few lines but a high character count may be rejected. To investigate this, we measure the average number of lines in a developer’s historically generated code suggestions (*developer_generated_lines*) and the average number of characters (*developer_generated_chars*).

Table 1. Overview of the features extracted to characterize developer Habits.

Dimension	Feature	Description	Source
Developer Code	<code>developer_preceding_lines</code>	Average lines of preceding context in all logs for a developer	Cbk
	<code>developer_preceding_chars</code>	Average characters of preceding context in all logs for a developer	Cbk
	<code>developer_generated_lines</code>	Average lines of generated code suggestions in all logs for a developer	Fdbk
	<code>developer_generated_chars</code>	Average characters of generated code suggestions in all logs for a developer	Fdbk
	<code>developer_modified_chars</code>	Average characters of modified code suggestions in all logs for a developer	Fdbk, Ref [13]
	<code>developer_modified_ratio</code>	Average ratio of modified accepted code suggestions in all logs for a developer	Fdbk, Ref [13]
	<code>developer_written_lines</code>	Average lines of written code in all logs for a developer	Log
	<code>developer_written_chars</code>	Average characters of written code in all logs for a developer	Log
	<code>developer_presented_time</code>	Average generation-to-presentation time of code suggestions in all logs for a developer	Fdbk
	<code>developer_accepted_time</code>	Average generation-to-presentation time of accepted code suggestions in all logs for a developer	Fdbk
	<code>developer_rejected_time</code>	Average generation-to-presentation time of rejected code suggestions in all logs for a developer	Fdbk
	<code>developer_generated_interval</code>	Average time interval between code generation triggers in all logs for a developer	Fdbk
	<code>developer_generated_counts</code>	Total number of historically generated code suggestions in all logs for a developer	Ref. [9]
Developer Acceptance Preference	<code>developer_accepted_length_ratio</code>	Average ratio of accepted length to total suggestion length in all logs for a developer.	Ref [44]
	<code>developer_accepted_ratio</code>	Average ratio of accepted to total generated code suggestions in all logs for a developer	Ref [44]
	<code>developer_accepted_counts</code>	Total counts of accepted code suggestions in all logs for a developer	Ref [44]
Developer Coding Environment	<code>developer_IDE_version_counts</code>	Total counts of unique IDE versions in all logs for a developer	Log
	<code>developer_IDE_type_counts</code>	Total counts of unique IDE types in all logs for a developer	Log
	<code>developer_dominant_IDE_version</code>	The most frequent used IDE version in all logs for a developer	Log
	<code>developer_dominant_IDE_type</code>	The most frequent used IDE type in all logs for a developer	Log

- **Modified Code length.** Both our preliminary developer feedback and literature review [13] suggest that developers sometimes accept code suggestions and then manually revise code that requires only minor corrections. If developers make fewer modifications to the generated code, it may indicate a higher level of satisfaction with the generated content and a greater likelihood of accepting the subsequent code. To investigate this, we measure the average levenshtein distance [23] between an accepted suggestion and its final version (i.e., the code captured when the tool triggers the next suggestion or the tool is closed), and the ratio of accepted suggestions that are subsequently modified (*developer_modified_ratio*) for a given developer. We adopt this definition because our goal is to characterize how developers modify an suggestion in the immediate coding context, rather than to evaluate long-term code evolution outcomes (e.g., at commit or merge time). Using the next-trigger boundary provides a consistent and tool-observable cutoff point that captures developers' short-term adjustments while minimizing interference from subsequent, potentially unrelated edits.
- **Written Code length.** The length of code a developer manually writes between suggestions may influence their subsequent acceptance decisions. To investigate this, we measure the average lines and characters of written code (*developer_written_lines* and *developer_written_chars*) for a given developer.
- **Generation Time.** In preliminary developer feedback, a developer mentions that slowness is the reason for rejecting code suggestions. Accepted and rejected code suggestions may have different generation time. To investigate this, we measure generation time for all presented, accepted, and rejected code suggestions for a given developer (*developer_presented_time*, *developer_accepted_time* and *developer_rejected_time*).
- **Generation Interval.** In the developer feedback, several developers reject suggestions if they are triggered too frequently, as constant interruptions can break their concentration. Therefore, we measure the average time elapsed between consecutive calls for code generation for a given developer (*developer_generated_interval*).
- **Generation Counts.** Prior work has found that past developer engagement with intelligent tools predicts future adoption [9]. A rich interaction history may reflect a developer's confidence in the AI-assisted programming tool, which may increase the probability that they will accept future code suggestions. We measure the total number of historically generated code suggestions (*developer_generated_counts*) for a given developer.

Developer Acceptance Preference Features: Prior work has found that a contributor's history of accepted pull requests is a strong predictor of future acceptance [41, 44]. Similarly, developer's past behavior with generated code suggestions may predict their future decisions. We measure

the average accepted length ratio (*developer_accepted_length_ratio*); the ratio of accepted to total generated code suggestions (*developer_accepted_ratio*); and the total number of accepted suggestions (*developer_accepted_counts*).

Developer Coding Environment Features: The version of the underlying generative model may impact the quality of code suggestions and, consequently, developer acceptance. The most common IDE type is VS Code, and there are other IDE types inside the company. To characterize programming environment for a given developer, we measure the total number of unique IDE release versions and type names (*developer_IDE_version_counts* and *developer_IDE_type_counts*); the most frequent used IDE version and type (*developer_dominant_IDE_version* and *developer_dominant_IDE_type*).

3.2.2 Project Preference Features. Project preference features analyze historical characteristics of a project. For example, developers may like to accept generated suggestions in early project stages, as these situations increase the need for coding guidance. Same to developer habit features, we also adopt a 7-day window when calculating project preference features. These features are also divided into three aspects, including project code features, project acceptance preference features and project programming environment features.

Project Code Features:

- **Preceding Comment Length.** The more comments indicate stricter programming standards [42], which may influence the acceptance of generated code suggestions. We measure the average lines of comment per 1000 lines for a given project (*project_comment_lines*).
- **Preceding Context Length.** Ross et al. [34] found that when project context is limited, developers may be in exploration mode and more likely to accept code suggestions. Analogous to developer preceding context features, we also measure the average lines and characters of preceding context for a given project (*project_preceding_lines* and *project_preceding_chars*).
- **Generated Code Length.** Analogous to developer generated code features, we also measure the average lines and characters of historically generated code suggestions for a given project (*project_generated_lines* and *project_generated_chars*).
- **Generation Time.** Analogous to developer generation time features, we also measure the generation time for all presented, accepted, and rejected code suggestions for a given project (*project_presented_time*, *project_accepted_time* and *project_rejected_time*).

Project Acceptance Preference Features: Inspired by the finding that a project's history of accepting contributions is a strong predictor of its future behavior [44], we introduce Project Acceptance Preference Features. Analogous to the developer acceptance features that capture individual habits, these project-level metrics quantify how well generated suggestions align with a project's collective standards and needs. We measure the average accepted length ratio (*project_accepted_length_ratio*); the ratio of accepted to total generated code suggestions (*project_accepted_ratio*); and the total number of accepted suggestions (*project_accepted_counts*).

Project Coding Environment Features: Analogous to developer programming environment features, in order to describe project adaptation with the evolution of development environments, we measure the total number of unique IDE versions and types (*project_IDE_version_counts* and *project_IDE_type_counts*); the most frequent used IDE version and type (*project_dominant_IDE_version* and *project_dominant_IDE_type*).

3.2.3 In-situ Context Features. Unlike developer habit and project preference features that are aggregated from historical data, in-situ features capture a snapshot of the immediate circumstances surrounding a single generation event. They focus on the specific context of the generated code suggestion, the characteristics of the suggestion itself, and the coding environment at that precise moment. These features are designed from three aspects:

Table 2. Overview of the features extracted to characterize Project Preferences.

Dimension	Feature	Description	Source
Project Code Features	project_comment_lines	Average lines of comment per 1000 lines in all logs for a project	Ref [42]
	project_preceding_lines	Average lines of preceding context in all logs for a project	Ref [34]
	project_preceding_chars	Average characters of preceding context in all logs for a project	Ref [34]
	project_generated_lines	Average lines of generated code suggestions in all logs for a project	Fdbk
	project_generated_chars	Average characters of generated code suggestions in all logs for a project	Fdbk
	project_written_lines	Average lines of written code suggestions in all logs for a project	Log
	project_written_chars	Average characters in written code suggestions in all logs for a project	Log
	project_presented_time	Average generation-to-presentation time of code suggestions in all logs for a project	Fdbk
	project_accepted_time	Average generation-to-presentation time of accepted code suggestions in all logs for a project	Fdbk
	project_rejected_time	Average generation-to-presentation time of rejected code suggestions in all logs for a project	Fdbk
Project Acceptance Preference	project_accepted_length_ratio	Average ratio of accepted length to total suggestion length in all logs for a project.	Ref [44]
	project_accepted_ratio	Average ratio of accepted to total generated code suggestions in all logs for a project	Ref [44]
	project_accepted_counts	Total counts of accepted code suggestions in all logs for a project	Ref [44]
Project Coding Environment	project_IDE_version_counts	Total counts of unique IDE versions in all logs for a project	Log
	project_IDE_type_counts	Total counts of unique IDE types in all logs for a project	Log
	project_dominant_IDE_version	The most frequent used IDE version in all logs for a project	Log
	project_dominant_IDE_type	The most frequent used IDE type in all logs for a project	Log

Preceding Context: The preceding context is defined as all content within the file before the point of code generation. Inspired by our industry partner’s Circuit Breaker, this set of features is extracted by analyzing this context, capturing metrics such as the number of lines (*in-situ_preceding_lines*), number of characters (*in-situ_preceding_chars* and *in-situ_preceding_last_line_chars*), comment length (*in-situ_preceding_comment_chars*), and whether the context ends in an incomplete code block (*in-situ_preceding_is_incomplete*). The goal is to characterize the preceding content to understand its impact on the final acceptance outcome.

Generated Code Suggestion: Generated code suggestion itself may be of great importance. To investigate this, we measure the length of the generated code suggestion (*in-situ_generated_lines* and *in-situ_generated_chars*). Furthermore, we also consider the cosine similarity between generated code suggestion and preceding context (*in-situ_generated_similarity*).

In-situ Coding Environment: Previous works [45, 47] have suggested that developers are more willing to accept code outside working hours and have higher adoption rates when using certain IDE. Furthermore, some developers set the max line or char limit of generated code suggestions. If the actual length of generated code suggestions is much different from developers’ setting, they may have lower acceptance probability. In-situ coding environment may also influence on the acceptance of developer’s decision. Therefore, we measure the generation time of in-situ code suggestion (*in-situ_generation_time*); whether the generation timestamp is in work day (*in-situ_is_workday*); the time period of the generated timestamp (*in-situ_time_period*); the in-situ IDE (*in-situ_IDE_type* and *in-situ_IDE_version*); the max limit of generated code suggestion (*in-situ_max_generated_line* and *in-situ_max_generated_char*); the absolute delta between max generated length and actual one (*in-situ_abs_lines_delta* and *in-situ_abs_chars_delta*).

3.3 Significance Analysis

In the previous subsection, we design multi-dimensional features. However, not all of these features can effectively distinguish between accepted and rejected code suggestions. We therefore employ significance testing to isolate the most influential ones—those showing a significant statistical difference between accepted and rejected code suggestions.

3.3.1 Methodology. We perform statistical analyses to identify the significant features of accepted code suggestions compared with rejected ones, which consists of three steps:

Step 1: Dataset Construction. We construct a dataset from the developer-AI interaction logs provided by our industry partner, including 66,239 entries from 113 developers across 12 projects collected over six months. Then, we extract candidate features as illustrated in Section 3.2.

Table 3. Overview of In-situ Features.

Dimension	Feature	Description	Source
Preceding Context	in-situ_preceding_lines	Lines of preceding context of in-situ log	Cbk
	in-situ_preceding_chars	Characters of preceding context of in-situ log	Cbk
	in-situ_preceding_last_line_chars	The characters of the last line of the preceding context of in-situ log	Cbk
	in-situ_preceding_comment_chars	The characters of comments of the preceding context of in-situ log	Cbk
	in-situ_preceding_is_incomplete	Whether the last line of the preceding context is an incomplete code block	Cbk
Generated Code Suggestion	in-situ_generated_lines	Lines of the generated code suggestion of in-situ log	Fdbk
	in-situ_generated_chars	Characters of the generated code suggestion of in-situ log	Fdbk
	in-situ_generated_similarity	Cosine similarity between generated code and preceding context of in-situ log	Log
In-situ Coding Environment	in-situ_generation_time	Time to generate the code suggestion of in-situ log	Fdbk
	in-situ_is_workday	Whether the generation time is a workday of in-situ log	Ref [45, 47]
	in-situ_time_period	The time period of the generation time of in-situ log (eg., 12:00–18:00)	Ref [45, 47]
	in-situ_IDE_type	The IDE type of in-situ log	Log
	in-situ_IDE_version	The IDE version of in-situ log	Log
	in-situ_max_generated_line	Max line limit of generated code suggestions for in-situ log.	Log
	in-situ_max_generated_char	Max char limit of generated code suggestions for in-situ log.	Log
	in-situ_abs_lines_delta	Absolute delta between the developer-set maximum and actual generated lines.	Log
	in-situ_abs_chars_delta	Absolute delta between the developer-set maximum and actual generated characters.	Log

Step 2: Testing Statistical Significance. According to previous work [21], we apply the Mann-Whitney U test with a 95% confidence level (i.e., $\alpha = 0.05$) [29], so as to test the statistical difference between the features of accepted and rejected code suggestions. We use this non-parametric test because we cannot assume a normal distribution for our features. A feature is considered statistically significant if its p-value, adjusted for multiple comparisons, is less than 0.05.

Step 3: Testing Practical Significance. While statistical significance verifies whether a difference exists, we also test for practical significance using Cliff’s delta [21] to estimate the magnitude of the difference (i.e., effect size). The value of Cliff’s delta (d) ranges from -1 to +1; a positive value indicates that the feature’s values are typically greater for accepted suggestions than for rejected ones, while a negative value implies the opposite. This directional difference is noted as "Greater" or "Less" in the significance tables. For instance, a "Greater" result for the *developer_accepted_ratio* feature indicates that accepted suggestions are characterized by a higher average ratio than rejected ones. Following prior work [21], we consider a feature to have practical significance if $|d| > 0.147$. Finally, a feature is considered significant if it is both statistically and practically significant. Note that this step focuses on identifying individually significant features. We explicitly address feature collinearity and redundancy in Section 4.1 to derive the final independent feature set for prediction.

3.3.2 Findings. Our analysis reveals that features related to developer habits and project preferences are the most significant features in code suggestion acceptance. In contrast, many in-situ features show limited statistical significance in our analysis. In the following, we discuss the significance of each feature dimension in more detail, with summaries provided in Tables 4, 5, and 6.

Accepted suggestions are linked to developers and projects that possess a more positive acceptance history. At the developer level, accepted suggestions are characterized by a higher developer acceptance rate (*developer_accepted_ratio*) and a greater number of previously accepted suggestions (*developer_accepted_counts*), as shown in Table 4. This suggests a positive feedback loop: satisfaction with the tool leads to continued engagement and trust. Similarly, at the project level, a project’s collective acceptance history is significantly important (Table 5). Accepted suggestions are typically found in projects where previous suggestions are also frequently accepted, as indicated by higher *project_accepted_ratio* and *project_accepted_counts*. This points to a strong alignment between the AI-assisted programming tool’s outputs and the project’s specific requirements.

Accepted code suggestions are linked to developers who experience longer average suggestion generation time. Interestingly, accepted code suggestions correspond to higher average generation times (*developer_presented_time* and *developer_accepted_time*). While counterintuitive from a user experience perspective, this could indicate that when the model consistently takes

Table 4. Overview of developer Habit Feature Significance

Significance	Feature Category	Feature Name	Accepted vs. Rejected
Significant	Developer Code	developer_generated_interval	Greater
		developer_generated_lines	Greater
		developer_generated_chars	Greater
		developer_presented_time	Greater
		developer_accepted_time	Greater
	Developer Acceptance Preference	developer_accepted_length_ratio	Greater
		developer_accepted_ratio	Greater
		developer_accepted_counts	Greater
Insignificant	Developer Code	developer_preceding_lines	–
		developer_preceding_chars	–
		developer_written_lines	–
		developer_written_chars	–
		developer_modified_chars	–
		developer_modified_ratio	–
		developer_rejected_time	–
		developer_generated_counts	–
	Developer Coding Environment	developer_IDE_version_counts	–
		developer_IDE_type_counts	–
		developer_dominant_IDE_version	–
		developer_dominant_IDE_type	–

Table 5. Overview of Project Preference Feature Significance

Significance	Feature Category	Feature Name	Accepted vs. Rejected
Significant	Project Code	project_preceding_lines	Less
		project_preceding_chars	Less
		project_presented_time	Greater
		project_accepted_time	Greater
	Project Acceptance Preference	project_accepted_length_ratio	Greater
		project_accepted_ratio	Greater
		project_accepted_counts	Greater
Insignificant	Project Code	project_comment_lines	–
		project_generated_lines	–
		project_generated_chars	–
		project_written_lines	–
		project_written_chars	–
		project_rejected_time	–
	Project Coding Environment	project_IDE_version_counts	–
		project_IDE_type_counts	–
		project_dominant_IDE_version	–
		project_dominant_IDE_type	–

more time for inference, it produces higher-quality, more relevant code when given more time for inference, which developers are ultimately more willing to accept despite the delay.

Accepted code suggestions are linked to developers who maintain longer intervals between triggers. We found that accepted suggestions are characterized by a longer average time interval between suggestion triggers (*developer_generated_interval*). This may be because frequent, unsolicited suggestions can disrupt a developer’s concentration and workflow. Longer pauses between suggestions may reduce this interference, allowing developers to more thoughtfully consider and accept the code proposed by the model.

Accepted code suggestions usually have smaller preceding contexts at the project level than rejected ones. In comparison to rejected code suggestions, accepted suggestions are in projects which have smaller preceding context (*project_preceding_lines* and *project_preceding_chars*). This scenario may occur in the early stages of a new feature or file (e.g., initializing a class, setting up a module). In these situations, developers may be more open to assistance to bootstrap their work or explore implementation ideas.

In-situ features are nuanced and hard to measure effectively. While many in-situ features appear statistically insignificant (Table 6), this does not diminish their real-world importance. For instance, the similarity between a suggestion and the preceding context (*in-situ_generated_similarity*) is not significant. This may be because developers often seek suggestions that are causally related to the preceding code (i.e., what should come *next*), not merely similar to it. This causal relationship is semantically complex and difficult to measure with simple metrics like cosine similarity. The most critical factor—the match between the generated code and the developer’s ultimate intent—is unobservable at the moment of generation, making it challenging to identify strong, predictive in-situ signals from logs alone. Indeed, prior work [9, 26, 27] confirms that one of the strongest predictors of acceptance is the semantic similarity between the generated suggestion and the one the developer eventually writes, a metric that is, by definition, unavailable at prediction time.

Impact of Model and IDE Version Updates on Acceptance. The only significant in-situ feature is the IDE version (*in-situ_IDE_version*), with accepted code suggestions occurring in older versions. As IDE version serves as a proxy for the underlying model version in our deployment, this pattern likely reflects model-version effects. Our industry partner explains that as the IDE and its associated libraries are updated over time, the model, trained on older data, may generate less effective or relevant code in newer IDE versions, leading to a poorer user experience and lower acceptance rates. Consequently, the model aligns better with older IDE versions, which explains why accepted suggestions are characterized by older IDE versions.

Answer to RQ₁. We find significant differences related to developer habits and project preferences between accepted and rejected code suggestions. At the developer level, accepted suggestions are characterized by the developer’s significantly higher historical acceptance count and ratio, and longer generation intervals. At the project level, they are characterized by the project’s significantly higher historical acceptance count and ratio, and a shorter preceding code context. In contrast, most in-situ features show limited statistical significance, likely because the most critical factor—alignment with a developer’s true intent—is unobservable at generation time. However, we find that accepted code suggestions occurring older IDE versions, suggesting the model has become less adapted to evolving coding corpora, which may degrade the user experience over time.

4 RQ₂: Code Suggestion Acceptance Prediction

Building on the significant features identified in RQ₁ across developer habits, project preferences, and in-situ context, RQ₂ turns these signals into a prediction task. We first distill a compact predictive set via correlation analysis, then design *CSAP* to integrate these features, and finally evaluate it against strong baselines with feature contribution analysis.

Table 6. Overview of In-situ Feature Significance

Significance	Feature Category	Feature Name	Accepted vs. Rejected
Significant	In-situ Coding Environment	in-situ_IDE_version	Less
Insignificant	Preceding Context	in-situ_preceding_lines	–
		in-situ_preceding_chars	–
		in-situ_preceding_last_line_chars	–
		in-situ_preceding_comment_chars	–
		in-situ_preceding_is_incomplete	–
	Generated Code Suggestion	in-situ_generated_lines	–
		in-situ_generated_chars	–
		in-situ_generated_similarity	–
	In-situ Coding Environment	in-situ_generation_time	–
		in-situ_is_workday	–
		in-situ_time_period	–
		in-situ_IDE_type	–
		in-situ_max_generated_char	–
		in-situ_max_generated_line	–
		in-situ_abs_lines_delta	–
		in-situ_abs_chars_delta	–

Table 7. Independent Features Used for Prediction

Feature Category	Feature Name
Developer Habit Features	developer_generated_interval
	developer_generated_lines
	developer_generated_chars
	developer_presented_time
	developer_accepted_time
	developer_accepted_ratio
	developer_accepted_counts
Project Preference Features	project_preceding_lines
	project_presented_time
	project_accepted_ratio
	project_accepted_counts
In-situ context Features	in-situ_IDE_version

4.1 Correlation Analysis

4.1.1 Feature Correlation Analysis. Significant features may include highly correlated ones, and directly inputting highly correlated features into models may bias results. We therefore compute Spearman rank correlations between feature pairs [21]. Under the premise of $p\text{-value} < 0.05$ (statistical significance), we treat $|\rho| > 0.8$ as strong correlation and keep a single representative feature per correlated cluster to reduce redundancy.

Our analysis revealed: (1) among developer habit features, only *developer_accepted_length_ratio* and *developer_accepted_ratio* have $|\rho| > 0.8$; we retain *developer_accepted_ratio*. (2) Among project preference features, *project_preceding_lines* and *project_preceding_chars* has $|\rho| > 0.8$; we retain *project_preceding_lines*. (3) *project_accepted_time*, *project_accepted_length_ratio*, and *project_accepted_ratio* are mutually correlated ($|\rho| > 0.8$); we retain *project_accepted_ratio*. All other features without high correlation are kept.

4.1.2 Independent Features for Prediction. After applying significance analysis and correlation pruning, we derive the final feature set for prediction. As detailed in Table 7, this set comprises features from three key categories: developer habit, project preference and in-situ context.

4.2 Method Design

We propose *CSAP* (Code Suggestion Acceptance Prediction), which formulates acceptance prediction as binary classification in Fig. 1. *CSAP* combines three sources: developer habit, project preference and in-situ context characteristics. In the training phase, the input features are first normalized and concatenated to form an M -dimensional vector, where M denotes the total number of features. This vector is then fed into a simple fully connected neural network, which uses a sigmoid function to map it to a probability. If the probability is larger or equal to the threshold, the generated code suggestion is predicted as accept, otherwise it is predicted as rejected. We discuss the parameter setting in the subsection 4.3.3. In the testing phase, we extract the same features and predict acceptance with the trained model.

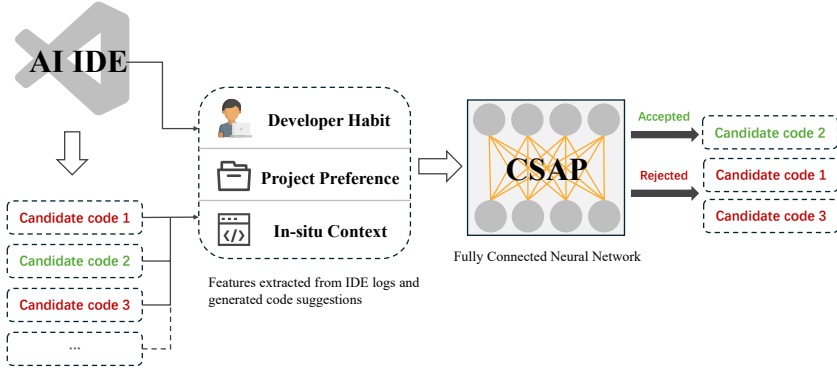


Fig. 1. Overview of our designed method *CSAP*

Previous work on GitHub Copilot found that developers rejected approximately 70% of the suggestions shown to them [47]. In the real developer-AI interaction logs provided by our industry partner, the majority of code suggestions are also rejected. Due to the imbalanced distribution of samples in the dataset, *CSAP* employs the class-balanced binary cross-entropy as the loss function and utilizes the AdamW optimizer [28] for gradient descent to train the predictive model. More specifically, Equation 1 defines the class-balanced binary cross-entropy. Here, $y_i \in \{0, 1\}$ is the ground-truth label (1=accept, 0=reject), $\hat{y}_i$ is the model score before the sigmoid, and $\sigma(\cdot)$ maps the score to a probability. The weights w_+ and w_- are the proportion of negative samples and positive samples, which are set inversely to class frequencies so that the minority class contributes more to the loss, mitigating the accept/reject imbalance. When accepted samples are rarer, w_+ becomes larger, increasing the penalty for missing accepted cases and improving probability calibration.

$$L_{\text{weight}} = -\frac{1}{N} \sum_{i=1}^N \left[w_+ y_i \log(\sigma(\hat{y}_i)) + w_- (1 - y_i) \log(1 - \sigma(\hat{y}_i)) \right] \quad (1)$$

4.3 Experimental Setup

4.3.1 Dataset Construction. To experimentally evaluate the *CSAP* prediction method, we construct an experimental dataset based on the code acceptance characteristic dataset containing 66,239 developer-AI interaction logs. We sort the dataset by time from earliest to latest, taking the first 80% of data as the training set and the last 20% as the test set. The training set has 52,991 entries,

which is used to train the model for acceptance prediction. The test set contains 13,248 entries, which is called the "imbalanced test set" in this paper.

To better validate the method's effectiveness, we construct a "balanced test set" by retaining all accepted instances from the imbalanced test set and subsampling an equal number of rejected instances.

4.3.2 Evaluation Metrics. According to previous work [1], we evaluate with Accuracy, Precision, Recall, F1-score. Accuracy is the fraction of predictions that match the true accept/reject label. Precision (accept) is the share of truly accepted items among those predicted as accepted, and Recall (accept) is the share of truly accepted items that we correctly predict as accepted; the F1-score (accept) is the harmonic mean of these two. We report Precision/Recall/F1 for the reject class analogously.

We also evaluate results by Binary Cross-Entropy (BCE), which is suggested by our industry partner. BCE measures the discrepancy between predicted probabilities and actual labels, thereby capturing the quality of the predictions and penalizing overconfident mistakes. The computation of Binary Cross-Entropy is as follows:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2)$$

Here, $\hat{y}$ is the vector of predicted probabilities for all data, y is the vector of labels for all data, N is the number of data items, y_i is the true label of the i -th sample, and $\hat{y}_i$ is the predicted probability that the i -th sample is accepted. For methods using LLM that output a definitive 'accept'/'reject' decision (a hard label) instead of a probability, we convert these outcomes to 0.999999 and 0.000001, respectively. This smoothing avoids $\log(0)$, approximates deterministic predictions as near-certain probabilities, and keeps BCE finite and comparable across methods. Values of Accuracy, Precision, Recall, F1-score are between 0 and 1, and larger values mean better performance. In contrast, larger Binary Cross-Entropy mean worse performance.

4.3.3 Parameter Settings. We sweep the decision threshold from 0.1 to 0.9 (Table 8). On the imbalanced test set, most metrics reach their best values to the right of 0.5 (≥ 0.6), and when a metric peaks elsewhere, 0.5 is within a small margin. On the balanced set, all metrics peak to the left of 0.5 (≈ 0.4). Since no single threshold dominates across both sets and 0.5 is consistently near-optimal, we retain 0.5 as a stable, distribution-agnostic decision threshold.

Table 8. CSAP Performance Metrics under Different Thresholds.

Metric		Probability Threshold								
		0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
Imbalanced	Accuracy	0.956	0.965	0.970	0.972	0.973	0.974	0.975	0.975	0.975
	Accept F1	0.793	0.828	0.848	0.857	0.861	0.864	0.866	0.864	0.862
	Reject F1	0.976	0.981	0.984	0.985	0.985	0.986	0.986	0.986	0.986
	Cross-Entropy	0.604	0.478	0.410	0.381	0.368	0.358	0.349	0.351	0.351
Balanced	Accuracy	0.917	0.920	0.922	0.924	0.922	0.921	0.921	0.916	0.910
	Accept F1	0.914	0.916	0.918	0.919	0.918	0.916	0.916	0.910	0.902
	Reject F1	0.921	0.923	0.926	0.928	0.927	0.925	0.926	0.921	0.916
	Cross-Entropy	1.142	1.109	1.071	1.049	1.071	1.093	1.093	1.159	1.246

4.4 Experimental Results

4.4.1 Prediction Performance. In order to evaluate the performance of *CSAP*, we compare *CSAP* against several baseline methods:

(1) Circuit Breaker Model. This is an in-production industrial filter. The model's approach is to predict the acceptance of code suggestions based on features extracted from the immediate preceding code context. These features primarily measure context size (e.g., counting the number of preceding lines), syntactic structure, and specific content types (e.g., checking if the last line is a comment). This method is a practical code acceptance prediction technique used internally by our industry partner.

(2) Direct LLM Call (Qwen2.5-Coder-32B). With the rise of powerful Large Language Models (LLMs), an intuitive baseline is to prompt an LLM with the same features used by *CSAP* to predict code acceptance directly. To structure the input, we designed a four-part prompt: (i) instructions telling the model to predict code acceptance, (ii) definitions of unique start/end markers for each feature, (iii) a rule that limits the output to either "accept" or "reject", and (iv) the actual feature values wrapped in their markers. This method treats the task as a general, in-context learning problem. We use it as a strong baseline to demonstrate the effectiveness of our specialized *CSAP* model, which is designed to handle structured numerical features. For this baseline, we use *Qwen2.5-Coder-32B*, a model deployed in production by our industry partner.

Table 9. *CSAP* and Baseline Performance on Imbalanced and Balanced Test Sets.

Dataset	Method	Acc.	Accept			Reject			Cross-Entropy
			Prec.	Rec.	F1	Prec.	Rec.	F1	
Imbalanced	<i>CSAP</i>	0.973	0.858	0.864	0.861	0.986	0.985	0.985	0.368
	Direct LLM Call	0.864	0.075	0.038	0.051	0.904	0.951	0.927	1.881
	Circuit Breaker	0.574	0.041	0.153	0.064	0.874	0.618	0.724	5.891
Balanced	<i>CSAP</i>	0.922	0.977	0.864	0.917	0.878	0.979	0.926	1.082
	Direct LLM Call	0.493	0.421	0.038	0.070	0.496	0.948	0.651	7.006
	Circuit Breaker	0.384	0.284	0.153	0.199	0.420	0.614	0.499	8.514

Table 9 summarizes *CSAP* and baseline results on the metrics in Section 4.3.2. Compared to the Circuit Breaker model and the direct LLM call, *CSAP* achieves significant accuracy improvements over baselines on both imbalanced (+12.6-69.5%) and balanced (+87.0-140.1%) datasets. Both baseline models perform poorly, particularly on the balanced test set, demonstrating their inability to handle class imbalance and effectively predict code acceptance. In contrast, *CSAP* delivers the highest accuracy and lowest cross-entropy, indicating superior calibration and discrimination for accepted code.

Beyond the improvements in predictive accuracy, *CSAP* demonstrates superior inference efficiency. To reflect real-world usage, we measured latency on the imbalanced test set by processing each sample sequentially within the same experimental client environment. *CSAP* achieves an average inference latency of 0.051 ms, representing an approximate 41% speedup over the Circuit Breaker baseline (0.086 ms) and an approximate 21,078× speedup compared to the direct LLM API call (1.075 s), fully satisfying the instant response requirements for real-time code suggestions, thereby validating *CSAP* as a lightweight and efficient solution.

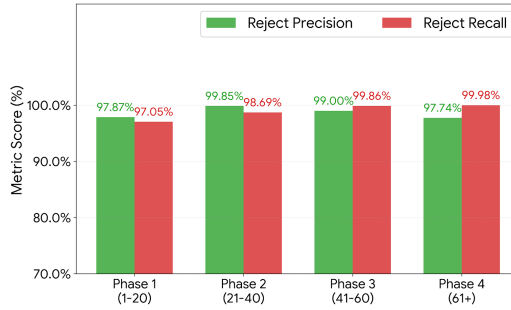


Fig. 2. Precision and recall of the rejection mechanism across IDE version phases

To examine whether CSAP’s filtering remains reliable as the development environment evolves, we evaluated its performance on the imbalanced test set across IDE version updates. To mitigate the noise inherent in industrial telemetry, we aggregated the data into four sequential phases based on IDE version intervals: Phase 1 (IDs 1-20), Phase 2 (IDs 21-40), Phase 3 (IDs 41-60), and Phase 4 (IDs 61+). Specifically, we analyzed the Precision/Recall of rejected code suggestions in Fig. 2. Reject Precision stays above 97.00% across all phases, indicating that when CSAP rejects, it is very likely to match what users would reject.

Table 10. CSAP Performance with Different Feature Sets on Imbalanced and Balanced Test Sets

Dataset	Method	Acc.	Accept			Reject			Cross-Entropy
			Prec.	Rec.	F1	Prec.	Rec.	F1	
Imbalanced	All Features	0.973	0.858	0.864	0.861	0.986	0.985	0.985	0.368
	w/o In-situ Context	0.899	0.480	0.739	0.582	0.971	0.916	0.942	1.397
	w/o Developer Habits	0.903	0.494	0.786	0.607	0.976	0.915	0.945	1.342
	w/o Project Preferences	0.965	0.806	0.842	0.822	0.983	0.978	0.981	0.480
Balanced	All Features	0.922	0.977	0.864	0.917	0.878	0.979	0.926	1.082
	w/o In-situ Context	0.826	0.895	0.739	0.810	0.778	0.914	0.840	2.399
	w/o Developer Habits	0.853	0.908	0.786	0.842	0.811	0.920	0.862	2.033
	w/o Project Preferences	0.909	0.973	0.842	0.903	0.861	0.977	0.915	1.251

4.4.2 Feature Contribution Analysis. To evaluate whether developer habit features, project preference features, and in-situ context features all contribute to CSAP’s prediction performance, we conducted three experiments under the same experimental settings as Section 4.3, each removing one aspect of features. We compared the results of these three experiments with the complete CSAP method to analyze the effectiveness of the three feature aspects in improving prediction performance. The experimental results are shown in Table 10.

The results show that after removing developer habit features, project preference features, and in-situ context features respectively, the model’s performance decreases on accuracy, precision, recall, and F1-score for both accepted and rejected code suggestion, and increases on binary cross-entropy, indicating that all proposed features contribute to improving the model’s prediction performance. Among these, removing in-situ context features results in the most significant degradation, highlighting their impact; developer habit features also matter, while project preferences have moderate effect.

Table 11. Individual feature ablation on Imbalanced and Balanced test sets (Δ = All Features – Model w/o feature).

Feature Name	Imbalanced		Balanced	
	Accuracy	Cross-Entropy	Accuracy	Cross-Entropy
developer_accepted_ratio	0.065	-0.898	0.039	-0.541
developer_accepted_counts	0.006	-0.082	0.004	-0.049
developer_generated_interval	0.003	-0.045	0.004	-0.055
developer_generated_lines	0.024	-0.116	0.003	-0.038
developer_generated_chars	0.002	-0.028	0.009	-0.126
developer_presented_time	-0.001	0.017	0.000	-0.005
developer_accepted_time	0.014	-0.187	0.002	-0.022
project_accepted_ratio	0.012	-0.161	0.008	-0.109
project_accepted_counts	0.002	-0.027	0.001	-0.011
project_preceding_lines	0.001	-0.019	0.001	-0.016
project_presented_time	-0.002	0.022	0.013	-0.180
in-situ_IDE_version	0.074	-1.029	0.096	-1.317

To explore the weight of each feature in the CSAP method and provide suggestions for model optimization, we conducted experiments removing each individual feature from the original CSAP method. Table 11 shows the difference between results with all features and results after removing each feature on the imbalanced and balanced test sets. Due to space constraints, the table only shows the results for Accuracy and Cross-Entropy. For Accuracy, positive values indicate that all features results outperform single-feature-removed results, and negative values indicate the opposite. For Cross-Entropy, negative values indicate that single-feature-removed results perform worse.

Our individual feature analysis, detailed in Table 11, reveals two key insights.

First, *in-situ_IDE_version* is unequivocally the most critical feature. As shown by the bolded values, its removal triggers the most significant performance degradation across nearly all metrics on both test sets. On the imbalanced set, its absence reduces Accuracy by 0.074, and yields a Cross-Entropy delta of -1.029. The impact is similarly dominant on the balanced set, where Accuracy drops by 0.096 and the Cross-Entropy delta reaches -1.317. This finding shows that developers are sensitive to the modernity of code suggestions, making it crucial for models to stay synchronized with evolving code corpora.

Second, historical acceptance ratios are powerful predictors of future behavior. The *developer_accepted_ratio* is particularly influential. Its removal leads to the second-largest drop in performance, decreasing accuracy by 0.065 and cross-entropy by -0.898 on the imbalanced set. This significant impact is mirrored on the balanced set, which sees a performance drop of 0.039 in accuracy and -0.541 in cross-entropy. Besides, *project_accepted_ratio* also contributes meaningfully, though to a lesser extent. Together, these features highlight that the history of accepting or rejecting suggestions is a strong indicator of future behavior.

Answer to RQ₂. We find that a personalized method to code acceptance prediction is highly effective. Our method, CSAP significantly outperforms both an in-production industrial filter and a large language model baseline. Feature analysis reveals that the most critical predictors, stemming from in-situ context, and developer habits, are the *in-situ_IDE_version*, and *developer_accepted_ratio*. These features highlight the importance of tailoring predictions to a developer's immediate environment and past behavior.

5 DISCUSSION

5.1 Implications

Our findings offer several practical and research implications for the future of AI-assisted programming tools.

Lightweight Post-Generation Filtering. Our findings indicate that a lightweight, post-generation filter provides the opportunity to improve the developer-perceived quality of an AI-assisted programming tool without modifying the underlying large language model. Specifically, our model CSAP can be trained and updated frequently on fresh interaction data, making the system highly responsive to evolving developer and project dynamics in a way that is not feasible for large, computationally expensive foundation models. Our results suggest that a hybrid architecture—pairing a powerful general-purpose generator with a nimble, personalized filter—presents a practical and cost-effective path toward more adaptive and efficient AI-assisted development.

Personalization as a First-Class Citizen. Our findings establish that personalization is critical for code suggestion acceptance. The significance analysis in RQ1 revealed that historical context—particularly a developer's and project's past acceptance behaviors—significantly distinguishes accepted suggestions from rejected ones (Section 3). Building on this, our predictive model, CSAP, confirmed these signals as powerful predictors (Section 4). Our feature contribution analysis reveals that historical acceptance ratios at both the developer and project levels are important. This evidence makes a strong case for treating personalization as a first-class citizen in AI-assisted programming. By learning from developer and project history, AI-assisted programming tools can evolve from generic generators to context-aware collaborators that provide more desired and less disruptive assistance. Future iterations could further empower users by allowing them to manually adjust the prediction threshold, balancing their personal tolerance for noise against the desire for more suggestions.

The Impact of Model Drift and Tool chain Evolution. Our findings indicate that the evolution of the development tool chain has a significant influence on code acceptance. Specifically, we observed that the IDE version, which serves as a proxy for the underlying model and its training data vintage, is a highly significant predictor of acceptance (Section 3). This suggests that as development environments and their associated libraries evolve, the scope of code generation expands. If the underlying large language model is not specifically adjusted to account for these changes, its suggestions may become outdated or incompatible with new APIs and syntax—even when trained on up-to-date corpora—leading developers to reject them. Practically, this finding implies that acceptance prediction models like CSAP can serve as a monitoring signal for timely model updates. A rising rejection rate predicted by CSAP on a new IDE version acts as an automated early warning system, allowing teams to proactively schedule model retraining based on detected misalignment between the model and the evolving environment rather than arbitrary time intervals.

Quality Over Speed: The Interplay of Length and Latency. Our findings challenge the conventional focus on minimizing latency, suggesting instead that developers prioritize suggestion

quality. This is supported by a twofold analysis. First, the significance tests in RQ1 show that accepted suggestions are associated with developers and projects that typically experience longer generation times and produce longer code (Table 4, Table 5). Second, our predictive model, *CSAP*, confirms that historical suggestion length (*developer_generated_lines*) and generation time for accepted suggestions (*developer_accepted_time*) are informative signals, as their removal degrades performance (Table 11). These results indicate that developers are willing to wait for more substantial, higher-quality generations. We therefore recommend that tool builders move beyond raw speed and instead implement adaptive latency budgets and learn personalized preferences for suggestion length to better align with developer intent.

Respecting Flow via Adaptive Intervals. Our findings on generation intervals suggest that constant interruptions harm acceptance. Specifically, we observed in RQ1 that accepted suggestions are characterized by significantly longer intervals between triggers (Tables 4). This indicates that frequent, unsolicited suggestions can disrupt a developer's concentration and workflow. Therefore, tool builders should move from maximizing frequency to *adaptive pacing*. Instead of triggering suggestions at every keystroke, systems should suppress suggestions during rapid coding bursts and learn the developer's unique rhythm. By offering help only during natural pauses, tools can minimize distraction and allow developers to thoughtfully consider the proposed code.

The Value of Fine-Grained Interaction Telemetry. Our findings indicate that access to fine-grained interaction telemetry is a prerequisite for building the next generation of personalized AI programming assistants. Specifically, we observed that the significant features distinguishing accepted from rejected suggestions are historical features derived from detailed interaction logs (Section 3). These logs enable us to quantify nuanced developer- and project-specific patterns, such as historical acceptance ratios. Without telemetry capturing every accept and reject decision along with its surrounding context, these critical predictive signals would remain invisible. Our results suggest that organizations aiming to optimize AI-assisted development should treat interaction data as a first-class asset. This data-driven approach allows for a shift from one-size-fits-all code generation to a more collaborative and less disruptive paradigm, where AI assistance is tailored to the specific needs of the developer and the context of their work.

Mitigating the Cold Start Problem with Project Wisdom. Our analysis identifies project-level acceptance history as a significant predictor of success. This offers a practical strategy to mitigate the "cold start" problem inherent in personalized systems. When new developers join a team, they lack personal interaction history, but our results show they can effectively "borrow" the collective patterns of their project. By prioritizing project-level signals—such as the project's historical acceptance ratio—AI tools can align suggestions with the team's specific coding standards and requirements. This approach enables high-quality, standard-compliant assistance from day one, significantly accelerating team onboarding without waiting for the accumulation of individual telemetry.

5.2 Threats to validity

Internal Validity. Firstly, this paper uses cosine similarity to calculate the *in-situ_generated_similarity* between the generated code suggestion and the preceding context. However, this metric mainly captures textual overlap rather than semantic alignment. In future work, we will use alternative similarity calculation methods. Secondly, the code suggestion is considered as accepted if the developer effectively retains some part of code suggestion before the tool's next trigger, typically a few minutes after the suggestion. However, the developer may delete the suggestion several days later. Nonetheless, the current acceptance definition captures the developer's immediate intent to engage with the tool, serving as the primary signal for interaction efficiency.

External Validity. Our study is conducted using telemetry data from a large technology company. Consequently, the developer population, coding practices, project types, and software stack may not be representative of other industrial settings or the open-source community. Regarding the scale, our dataset includes 66,239 developer-AI interaction logs from 113 developers across 12 projects. While the number of participating developers may not appear large and the number of projects is low in certain enterprise contexts, as the first in-depth quantitative study of its kind using fine-grained, proprietary interaction data, it provides an unprecedented depth of telemetry to validate personalized features. Besides, our proposed methodology and findings possess a degree of generalizability. Firstly, CSAP relies on universal IDE interaction features—such as context length, generation intervals, and historical acceptance rates, making the framework adaptable to other AI-assisted programming tools. Secondly, our findings indicate that historical interaction logs are important predictors of future acceptance. While replication on public repositories is infeasible due to the absence of such detailed telemetry (e.g., ghost text and real-time rejections), our work bridges this gap by demonstrating that leveraging fine-grained interaction telemetry is a viable and generalizable technical pathway for improving AI assistants.

Construct Validity. Our features are proxies for complex underlying phenomena, but there are some other features that have not been considered in this paper. For example, we use historical code metrics to represent a developer’s coding habits, but this is an incomplete view of their preferences and style. Furthermore, due to strict data privacy and protection policies, we are unable to access specific developer attributes (e.g., tenure, level, attitude) or project history metadata (e.g., file size, file age, churn). Future works may try to collect more information from other companies and explore how these unmeasured factors influence acceptance decisions.

6 CONCLUSION

AI-assisted programming tools are increasingly integrated into developer workflows, but their potential is often hindered by undesired suggestions that cause costly interruptions. Our empirical study on industrial data reveals that developer habits, project preferences, and in-situ context significantly distinguish accepted suggestions from rejected ones. Leveraging these insights, we proposed CSAP, a lightweight personalized filter that predicts acceptance before display. Evaluations show that CSAP significantly outperforms competitive baselines. To the best of our knowledge, as the first large-scale quantitative study of its kind, this work underscores the critical role of personalization in AI-assisted programming.

7 DATA AVAILABILITY

The source code and scripts used for analysis in this study are available online: <https://figshare.com/s/5e33393a73831c00a75d>. The dataset used for this research is internal to our industry partner and cannot be made publicly available due to a confidentiality agreement.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China Grants 92582204, 62577007 and 62572030, in part by the Fundamental Research Funds for the Central Universities. We also thank the anonymous reviewers for their constructive feedback that helped improve this paper.

References

- [1] Yuanrui Fan and· Xin Xia and· David Lo and·Shanping Li. 2018. Early prediction of merged code changes to prioritize reviewing tasks. *Empirical Software Engineering* 23 (2018), 3346–3393.

- [2] Anthropic. 2024. Claude 3.7 Sonnet and Claude Code. <https://www.anthropic.com/news/claude-3-7-sonnet>. Accessed: 2025-09-05.
- [3] Muhammad Ilyas Azeem, Sebastiano Panichella, Andrea Di Sorbo, Alexander Serebrenik, and Qing Wang. 2020. Action-based recommendation in pull-request development. In *Proceedings of the International Conference on Software and System Processes*. 115–124.
- [4] Gal Bakal, Ali Dasdan, Yaniv Katz, Michael Kaufman, and Guy Levin. 2025. Experience with GitHub Copilot for Developer Productivity at Zoominfo. *arXiv preprint arXiv:2501.13282* (2025).
- [5] Lingfeng Bao, Zhenchang Xing, Xin Xia, David Lo, and Shanping Li. 2017. Who will leave the company?: a large-scale industry study of developer turnover by mining monthly work report. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 170–181.
- [6] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [7] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2022. Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue* 20, 6 (2022), 35–57.
- [8] Adam Brown, Sarah D'Angelo, Ambar Murillo, Ciera Jaspán, and Collin Green. 2024. Identifying the factors that influence trust in AI code completion. In *Proceedings of the 1st ACM International Conference on AI-Powered Software*. 1–9.
- [9] Matteo Ciniselli, Luca Pascarella, Emad Aghajani, Simone Scalabrino, Rocco Oliveto, and Gabriele Bavota. 2023. Source code recommender systems: The practitioners' perspective. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2161–2172.
- [10] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [11] Nicole Davila, Igor Wiese, Igor Steinmacher, Lucas Lucio da Silva, André Kawamoto, Gilson José Peres Favaro, and Ingrid Nunes. 2024. An industry case study on adoption of ai-based programming assistants. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. 92–102.
- [12] Nicole Dresselhaus. 2025. Field Report: Coding in the Age of AI with Cursor. <https://drezil.de/Writing/coding-age-ai.html>
- [13] Daniel Erhabor, Sreeharsha Udayashankar, Meiyappan Nagappan, and Samer Al-Kiswani. 2025. Measuring the Runtime Performance of C++ Code Written by Humans using GitHub Copilot. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 596–596.
- [14] Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchun Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, et al. 2025. Trae Agent: An LLM-based Agent for Software Engineering with Test-time Scaling. *arXiv preprint arXiv:2507.23370* (2025).
- [15] Emanuela Guglielmi, Venera Arnoldova, Gabriele Bavota, Rocco Oliveto, and Simone Scalabrino. 2025. How do Copilot Suggestions Impact Developers' Frustration and Productivity? *arXiv preprint arXiv:2504.06808* (2025).
- [16] Daya Guo, Dejian Yang, Haowei Zhang, et al. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645 (Sep 2025), 633–638. <https://doi.org/10.1038/s41586-025-09422-z>
- [17] Vincent J Hellendoorn, Sebastian Proksch, Harald C Gall, and Alberto Bacchelli. 2019. When code completion fails: A case study on real-world completions. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 960–970.
- [18] Khairul Islam, Toufique Ahmed, Rifat Shahriyar, Anindya Iqbal, and Gias Uddin. 2022. Early prediction for merged vs abandoned code changes in modern code reviews. *Information and Software Technology* 142 (2022), 106756.
- [19] Rinkesh Joshi and Nafiseh Kahani. 2024. Comparative Study of Reinforcement Learning in GitHub Pull Request Outcome Predictions. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 489–500.
- [20] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2012. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering* 39, 6 (2012), 757–773.
- [21] SayedHassan Khatoonabadi, Diego Elias Costa, Rabe Abdalkareem, and Emad Shihab. 2023. On wasted contributions: Understanding the dynamics of contributor-abandoned pull requests—a mixed-methods study of 10 large open-source projects. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–39.
- [22] Htoo Htoo Sandi Kyaw, Shwe Thinzar Aung, Hnin Aye Thant, and Nobuo Funabiki. 2018. A proposal of code completion problem for java programming learning assistant system. In *Conference on Complex, Intelligent, and Software Intensive Systems*. Springer, 855–864.

- [23] Vladimir Iosifovich Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady* 10, 8 (1966), 707–710.
- [24] Jingxuan Li, Rui Huang, Wei Li, Kai Yao, and Weiguo Tan. 2021. Toward less hidden cost of code completion with acceptance and ranking models. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 195–205.
- [25] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustín Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [26] J. T. Liang, C. Yang, and B. A. Myers. 2024. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, Lisbon, Portugal, 1–13.
- [27] Xiao Long, Xin Tan, Yinghao Zhu, Jing Jiang, and Li Zhang. 2025. Understanding and Enhancing CS Students' Interaction Experience with AI Coding Assistant Tools. *ACM Trans. Softw. Eng. Methodol.* (Dec. 2025). <https://doi.org/10.1145/3785479> Just Accepted.
- [28] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=Bkg6UKwA->
- [29] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60.
- [30] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–16.
- [31] Nachiappan Nagappan and Thomas Ball. 2005. Use of relative code churn measures to predict system defect density. In *Proceedings of the 27th international conference on Software engineering*. 284–292.
- [32] Ruchika Pandey, Prabhat Singh, Raymond Wei, and Shailla Shankar. 2024. Transforming software development: Evaluating the efficiency and challenges of github copilot in real-world projects. *arXiv preprint arXiv:2406.17910* (2024).
- [33] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* 68, 2 (2025), 96–105.
- [34] Steven I Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D Weisz. 2023. The programmer's assistant: Conversational interaction with a large language model for software development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. 491–514.
- [35] Auste Simkute, Lev Tankelevitch, Viktor Kewenig, Ava Elizabeth Scott, Abigail Sellen, and Sean Rintel. 2025. Ironies of generative AI: understanding and mitigating productivity loss in Human-AI interaction. *International Journal of Human-Computer Interaction* 41, 5 (2025), 2898–2919.
- [36] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, Mingze Ni, and Li Li. 2023. Don't Complete It! Preventing Unhelpful Code Completion for Productive and Sustainable Neural Code Completion Systems. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 324–325. <https://doi.org/10.1109/ICSE-Companion58688.2023.00089>
- [37] Prathmesh Suryavanshi, Manohar Kapse, and Vinod Sharma. 2025. Integrating ChatGPT into Software Development: Valuating Acceptance and Utilisation Among Developers. *Australasian Accounting, Business and Finance Journal* 19, 1 (2025).
- [38] Zhaopeng Tu, Zhendong Su, and Premkumar Devanbu. 2014. On the localness of software. In *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*. 269–280.
- [39] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*. 1–7.
- [40] Chaozheng Wang, Junhao Hu, Cuiyun Gao, Yu Jin, Tao Xie, Hailiang Huang, Zhenyu Lei, and Yuetang Deng. 2023. How practitioners expect code completion?. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1294–1306.
- [41] Dandan Wang, Qing Wang, Junjie Wang, and Lin Shi. 2021. Accept or not? an empirical study on analyzing the factors that affect the outcomes of modern code review?. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 946–955.
- [42] Chuhan Wu, Fangzhao Wu, Yongfeng Huang, and Xing Xie. 2023. Personalized news recommendation: Methods and challenges. *ACM Transactions on Information Systems* 41, 1 (2023), 1–50.
- [43] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [44] Xunhui Zhang, Yue Yu, Georgios Gousios, and Ayushi Rastogi. 2022. Pull request decisions explained: An empirical overview. *IEEE Transactions on Software Engineering* 49, 2 (2022), 849–871.

- [45] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5673–5684.
- [46] Xiyu Zhou, Peng Liang, Beiqi Zhang, Zengyang Li, Aakash Ahmad, Mojtaba Shahin, and Muhammad Waseem. 2025. Exploring the problems, their causes and solutions of AI pair programming: A study on GitHub and Stack Overflow. *Journal of Systems and Software* 219 (2025), 112204.
- [47] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 21–29.